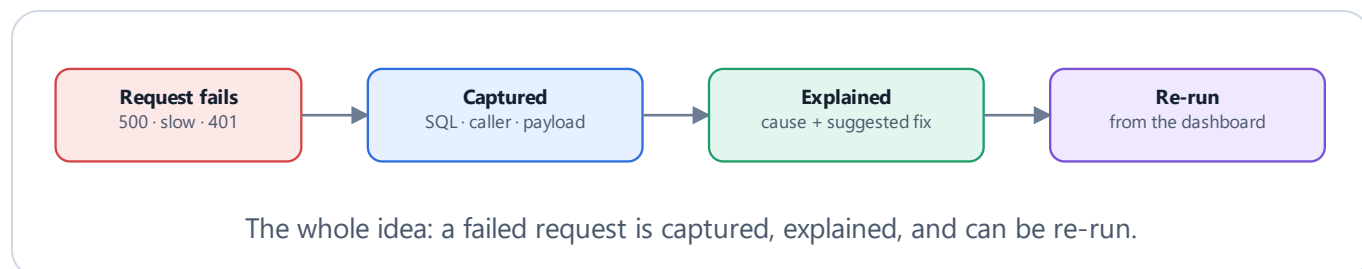


JSCoup

Capture API failures inside your Python app, explain each one automatically, and re-run it from a built-in dashboard, with a scoped gateway for outside callers.

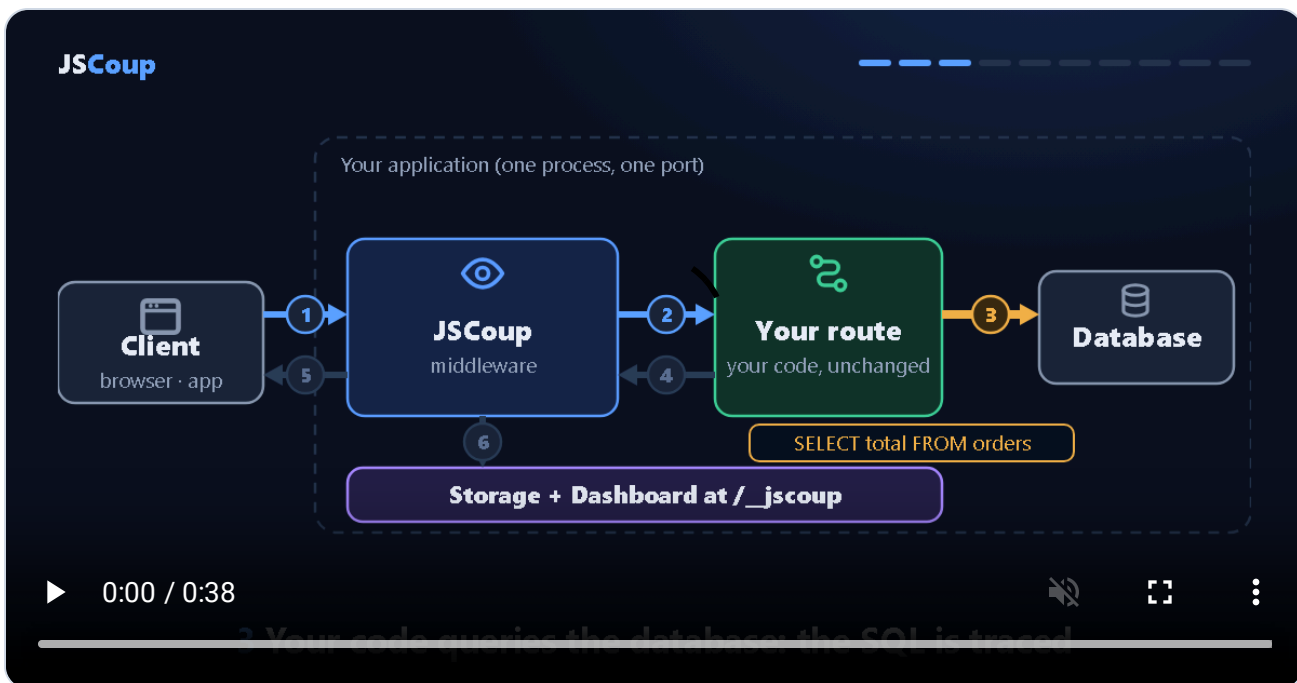


By Zaidon Aljbaae · MIT License · Python 3.9+ · Flask, FastAPI, Django, Lambda, Azure Functions, OCI Functions

1 Abstract

Production Python services fail in ways that are slow to diagnose. The traceback survives, but the SQL that was running, the request that triggered it and the user behind it usually do not. JSCoup keeps that context. It runs inside the application process, records each failure, slow call and (optionally) successful call with its surrounding evidence, classifies it with a rule-based diagnostic engine, and shows the result in a login-protected dashboard mounted on the application's own port.

The same discovered model of the API surface also powers a live tester, a published documentation view and an admin-curated gateway that hands scoped, expiring access to outside callers. Three tools that are usually separate (an error tracker, an API explorer and a small gateway) share one registry and one execution path.



Thirty-eight seconds: the middleware sees every call, reads your routes, flags the failing API, explains it, and lets you re-run it.

2 The problem

Tracebacks without context

The failing SQL, its parameters and the calling user are rarely in the log. Engineers rebuild them from surrounding lines, if those exist.

Documentation that drifts

A hand-edited spec is wrong the day a parameter is renamed, and it cannot show whether a call still works.

A gateway for three endpoints

Giving one partner scoped, revocable access should not need a separate infrastructure product.

All three share one prerequisite: a structured, in-process model of what the application exposes and what happened on each call. JSCoup builds that model once and reuses it.

3 What JSCoup is

A single Python package with no required dependencies. One object, `JSCoup`, is created once; `install(app)` wraps every route of a Flask, FastAPI or Django application and mounts the dashboard at `/__jscoup` on the same port.

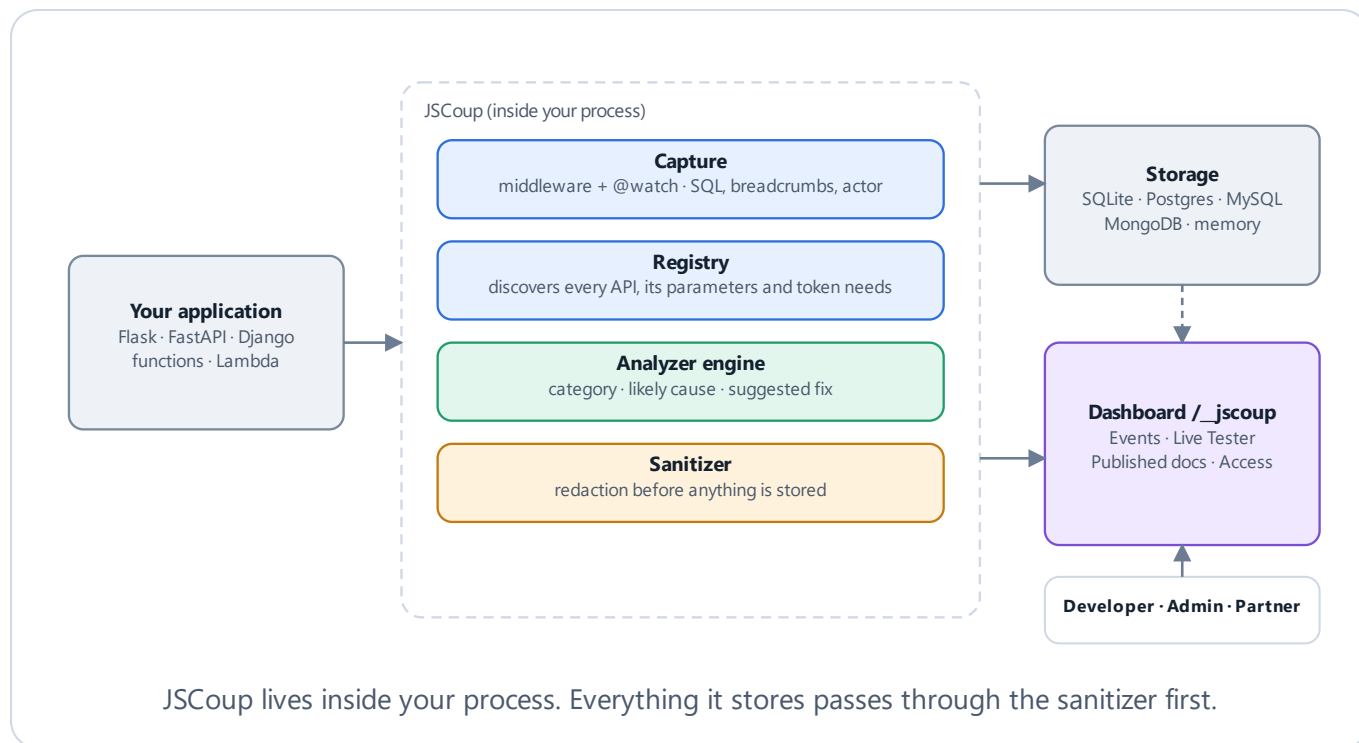
```
from jscoup import JSCoup

bl = JSCoup(service_name="orders-api",
            dashboard_username="admin", dashboard_password="change-me")
bl.watch_sqlite()      # optional: trace every sqlite3 statement
bl.install(app)        # Flask, FastAPI or Django, detected from the object

# open http://127.0.0.1:8000/__jscoup
```

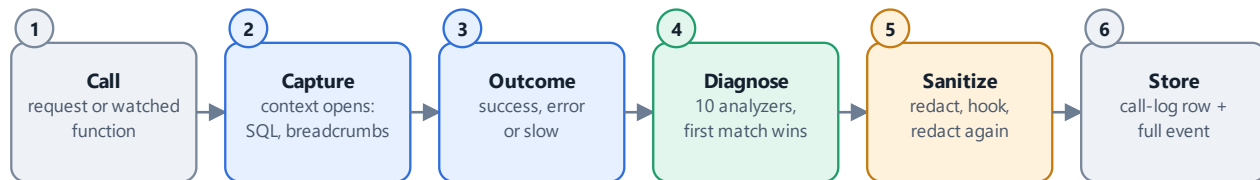
- **Capture:** errors, slow calls, N+1 query patterns, authentication failures, printed output (optional), with the exact source line of the failure.
- **Diagnose:** category, likely cause and suggested fix for each failure.
- **Replay:** re-run the real API from the dashboard, or from code with `invoke()` and `ainvoke()`.
- **Publish:** a documentation page with examples and a working "Try it".
- **Share:** gateway users with a password, an allow-list of APIs, an optional IP list and an expiring token.

4 Architecture at a glance



The **capture** layer is middleware plus optional decorators. The **registry** discovers the API surface from the running framework. The **analyzer engine** turns evidence into a diagnosis. The **sanitizer** removes secrets before storage. Storage is pluggable behind one interface, and the dashboard is a set of routes mounted into the same application.

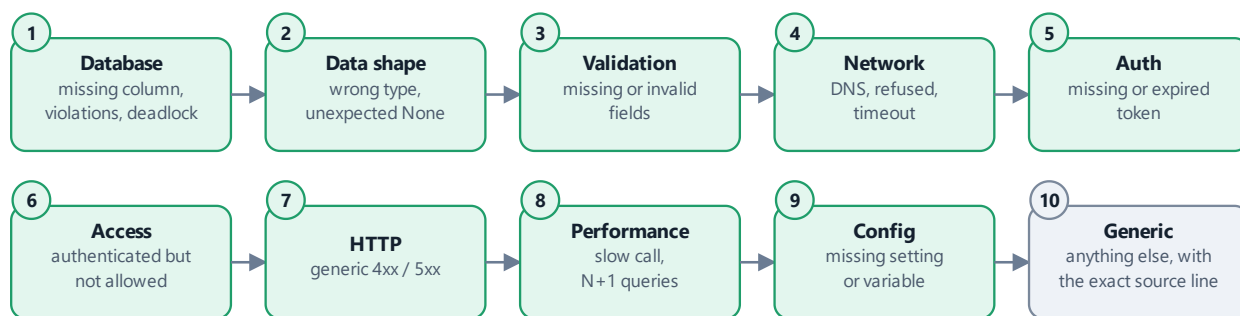
5 Anatomy of one call



1. A request (or a watched function, Lambda or Azure Function call) opens a **capture context** scoped with `contextvars`. SQL statements, breadcrumbs and optional stdout accumulate in it as the call runs.
2. When the call ends, an event is built: path, status, timing, actor, traceback and the exact offending source line.
3. The **analyzers** classify it (next section).
4. The event passes the **sanitizer**, then your optional `before_store` hook (which can rewrite or drop it), then the sanitizer again, because a hook may reintroduce content.
5. Two records are written: a lightweight **call-log row** (name, timing, ok or failed, never a body or a person) and, for a failure, a slow call, or when `capture_success` is on, the **full event**.

Background delivery is available for high traffic: a bounded queue with counters and `flush()` / `close()`. It is best effort and drops events when full rather than slowing requests.

6 The diagnosis engine



Ten analyzers in a fixed order. The first one that matches decides.

Diagnosis is rule-based, not statistical. Each analyzer looks for **structural evidence first**: the exact exception class, the driver module, the presence of a failed query. It falls back to message text only when nothing structural is available. This ordering avoids the classic mistake of classifying by a substring: a plain `TimeoutError` is not a database error, while a pool timeout from the SQL driver is.

You see	JSCoup reports
<code>UndefinedColumn</code> from the Postgres driver	Database, missing column, with the failing statement and its timing
401 on a route behind a token dependency	Auth, missing or expired token
Eight identical queries inside one request	Performance, N+1 pattern
An exception no rule recognises	Generic, with the exact file, line and source text

You can add your own analyzers, run before or instead of the built-in ones, with `JSCoup(analyzers=[...])`.

7 Discovery: one registry, three consumers



Installing JSCoup inspects every route once and records, per API, its parameters and whether it needs a token. Because discovery and replay share one code path, a call the Live Tester can make is a call the gateway can make, with the same parameter handling.

What is detected

- **Parameters:** location (path, query, form, body, file), required or optional, default, description, allowed values (`Literal` , `Enum`) and rules such as `pattern` or `ge` . Example values respect those rules and optional inputs are left empty.
- **Token needs:** for FastAPI this is read from FastAPI's own resolved dependency tree, so `Depends(get_current_user)` and anything behind it is understood. An `OAuth2PasswordBearer(auto_error=False)` is reported as *token optional*; a route with no security scheme needs no token.

- **Hidden inputs:** a class dependency such as `OAuth2PasswordRequestForm` contributes its own fields (`username` , `password` and four optional OAuth2 fields).

```
def current_user(token: str = Depends(oauth2)): ...      # needs a token

@app.post("/login")                                     # needs no token
def login(form: OAuth2PasswordRequestForm = Depends()): ...

@app.get("/items")                                       # token required
def items(q: Optional[str] = None, user=Depends(current_user)): ...
```

Flask uses `flask_parameter_validation` declarations, direct `request.args` / `get_json()` reads and auth decorators. Django uses its URL patterns and view signatures. A route added later is picked up before the next dashboard request.

8 The dashboard

Overview

Call totals, success and failure split, top failing APIs, retention window.

Events

Each failure with traceback, exact source line, SQL with timing, breadcrumbs, caller and diagnosis.

Live Tester

Every discovered API grouped by source file, with typed forms. Execute it and see the real response, and the diagnosis if it fails.

Published docs

Read-only groups with examples and a working "Try it", limited by their own IP policy. Export to Postman.

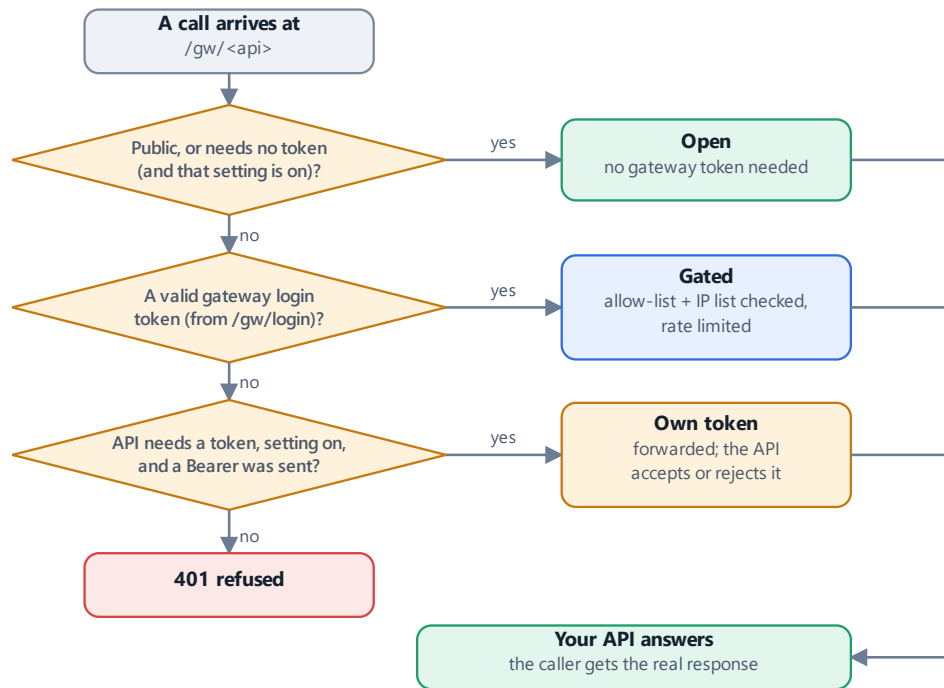
Access

Create gateway users, set allow-lists, mark APIs public, revoke.

Login

Username and password, logout, session expiry, optional IP allow-list. No credentials given means a random password printed once, never an open dashboard.

9 The API gateway



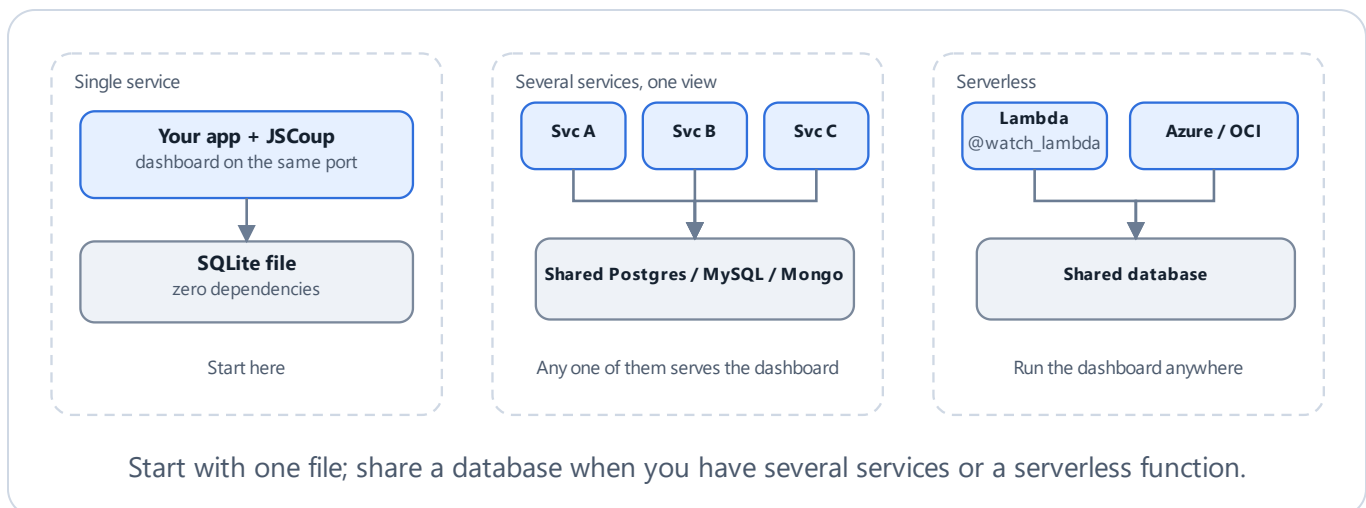
How a call to `/gw/<api>` is allowed through.

The gateway is deliberately small: it hands a known set of your own APIs to a few outside callers. An admin creates a user with a password, the APIs they may call and, optionally, an IP list. The user logs in at `POST /gw/login` and receives a short-lived token. JSCoup then calls the API on their behalf, so the caller never reaches the route directly and never sees a traceback or SQL.

Setting	Default	Effect
<code>gateway_enabled</code>	off	Turns the gateway on.
<code>gateway_open_tokenless_apis</code>	off	An API that needs no token of its own (such as a login route) is callable with no gateway token. An API that needs a token is never opened by it.
<code>gateway_forward_own_tokens</code>	off	An API that needs a token can be called by presenting that API's own token as the Bearer token. The API itself accepts or rejects it.
<code>redact_replay_response</code>	off	Mask token-like fields in the response a replay returns. Off means the caller gets the real answer, so a login route can hand out its token. Stored events are always redacted.
<code>allow_live_invoke</code>	off	Lets the dashboard execute APIs. Opt in.

Design rule. Detection never opens an API to the gateway by itself. Each opening is an explicit setting or an explicit admin choice.

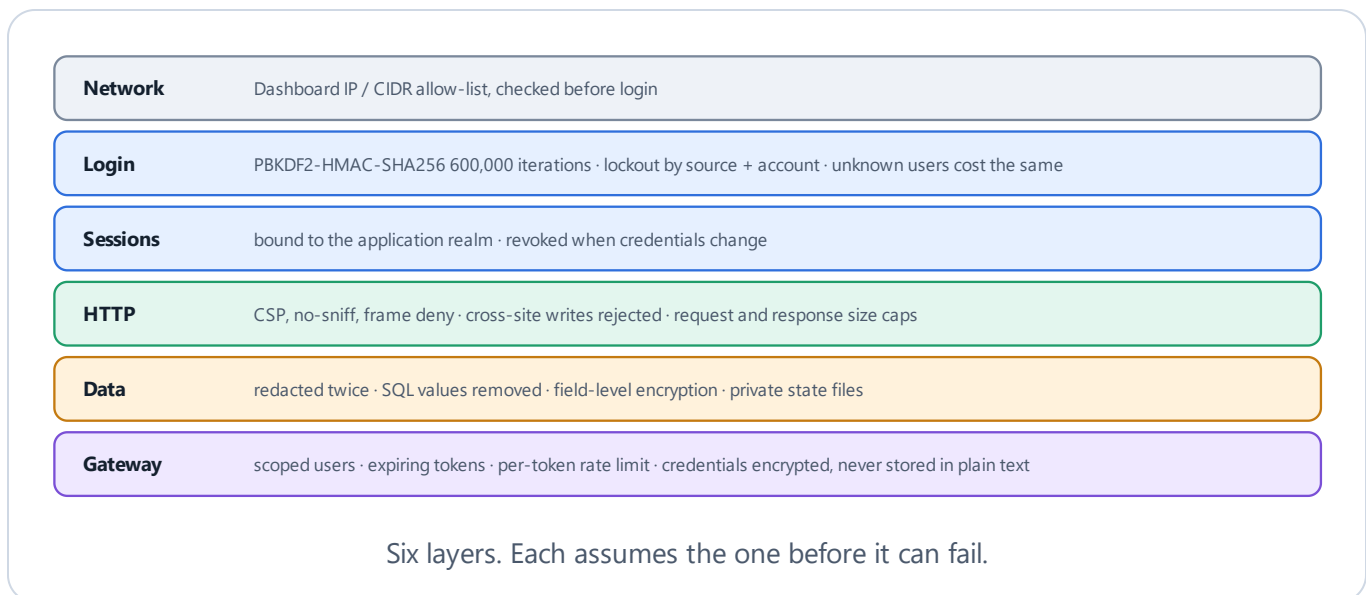
10 Storage and deployment shapes



Backend	Extra	Use when
SQLite (default)	none	One service, no shared infrastructure.
SQL (Postgres, MySQL, Oracle)	<code>jscoup[sql]</code>	Several workers or services sharing one view. An isolated schema keeps its tables apart from yours.
MongoDB	<code>jscoup[mongo]</code>	Your stack already uses MongoDB. Summaries run server-side.
Memory	none	Tests.

SSH-tunnelled database connections, connection files (JSON, YAML, INI) and field-level encryption at rest are supported. A serverless function has no disk for a dashboard, so it writes to a shared database and any other process serves the dashboard.

11 Security and privacy



A stack of small, separate controls rather than one big one. Secrets are removed **before** storage, not at display time, and the call log never holds a response body or a person.

Honest limits. JSCoup is a single-process library: background delivery is best effort and a crash can lose queued events; gateway rate limits are local, not distributed; Windows file permissions are the operator's job; the dashboard's Content-Security-Policy still allows inline scripts; and there has been no independent third-party audit. Put an edge gateway in front if you need distributed rate limiting, and measure sustained load on your own hardware.

12 Get started

```
pip install jscoup # core, zero dependencies
pip install "jscoup[fastapi,sql]" # + FastAPI adapter + Postgres/MySQL storage
```

Runnable examples for Flask, FastAPI and plain functions are in the [examples](#) folder. The [integration guide](#) covers each framework, serverless, database tracing, multi-service setups and common mistakes; the [API reference](#) lists every method and setting.

13 How it compares

	Hand-written OpenAPI	Hosted APM / error tracker	JSCoup
Documents the real endpoints	Until it drifts	No	Yes, discovered from the running app
Executes an endpoint from the docs	No	No	Yes
Explains why a call failed	No	Traceback	Category, cause, fix
Captures the failing SQL	No	Rarely	Statement, timing, call site
Scoped access for a partner	No	No	Built-in gateway
Data stays on your infrastructure	Yes	No (SaaS)	Yes
Cross-service analytics at scale	n/a	Yes	No: use it as a complement

14 Quality and verification

549 tests pass

0 failures. 5 tests need a live database and run in CI against real PostgreSQL and MongoDB.

Python 3.9 to 3.12

CI matrix, plus a job that boots eight processes against one fresh Postgres to catch start-up races.

Real applications

Run inside a FastAPI + Postgres platform and a large Flask multi-service project, with the failures they produced fixed in the library.

Clean install

The built wheel installs into an empty environment with no dependencies and passes `twine check`.

15 Ownership, license and citation

Copyright © 2026 **Zaidon Aljbaae**. Released under the [MIT License](#): use, copy, modify and distribute freely, keeping the copyright notice. Every source file carries an SPDX header and the package exposes `jscoup.__author__`. The name and logo are covered by a separate [trademark notice](#). Releases are built by a CI workflow and published to PyPI with signed provenance that links each package to this repository and commit.

```
@software{aljbaae_jscoup_2026,
  author  = {Aljbaae, Zaidon},
  title   = {JSCoup},
  version = {1.0.0},
  year    = {2026},
  url     = {https://github.com/zaidonaljbaae/JSCOUF}
}
```

JSCoup work paper, version 1.0.0. © 2026 Zaidon Aljbaae, MIT License. This paper describes the library's own design and documented behaviour; it is not an independent audit.